Lecture 8 — Advanced Strings: Eertree, Suffix Trees, Lyndon Words, Periodicity

> World Finals Level · Prerequisites: Year weeks 28–29 (hashing, KMP, Z, tries, suffix array intro), regional-09 (Manacher), nationals-06 (suffix array + LCP, suffix automaton) · Estimated time: 2 × 90 min lecture + 3 h lab

1. Overview

By this point you have a general-purpose string toolkit: hashing, prefix function, Z-function, Aho–Corasick, suffix arrays and suffix automata. World Finals and Codeforces Div 1 string problems usually need one more ingredient: a *structural theorem* about strings that bounds the number of interesting objects and tells you how to enumerate them. This lecture covers four such theorems and the algorithms built on them:

1. **At most n distinct palindromes** → the palindromic tree (eertree), and **palindromic suffixes form O(log n) arithmetic progressions** → O(n log n) palindromic DPs.
2. **The suffix tree** as a compressed trie, obtained cheaply as the suffix-link tree of the suffix automaton of the reversed string.
3. **Unique Lyndon factorisation** (Chen–Fox–Lyndon) → Duval's algorithm, minimal rotations.
4. **Periodicity**: border/period duality, the Fine–Wilf lemma, and the **Runs theorem** (fewer than n maximal repetitions) → enumerating all runs via Lyndon roots.

Every implementation in §7 was tested against brute force on thousands of random strings over alphabets of size 1–3 (small alphabets maximise repetition and hence bugs).

2. The palindromic tree (eertree)

2.1 The key lemma

Lemma. A string of length n has at most n distinct non-empty palindromic substrings.

Proof. Build the string one character at a time. Suppose appending c to s creates new palindromes. Every new palindrome is a suffix of s·c. Let P be the longest palindromic suffix of s·c and Q any shorter palindromic suffix. Since P is a palindrome and Q is a suffix of P, the reverse of Q, which is Q itself, is a prefix of P; so Q occurs ending before the last position and is not new. Hence at most one new palindrome (P) per character. ∎

So a structure with one node per distinct palindrome has linear size.

2.2 Structure

The eertree has two roots: an imaginary node of length −1 and the empty palindrome of length 0. Every other node is a distinct palindrome. An edge v —c→ u means u = c v c; children of the −1 root are single characters (adding c on both sides of a "length −1 string" yields c). Each node v has a **suffix link** to the longest proper palindromic suffix of v; single characters and the empty string link to the empty node, and the empty node links to the −1 root. The algorithm also maintains `last`, the node of the longest palindromic suffix of the current prefix.

2.3 Appending a character

To append s[i] = c, walk from `last` along suffix links to the first node X such that s[i − |X| − 1] = c (the −1 root always succeeds, since then i − |X| − 1 = i). Then cXc is the longest palindromic suffix of the new prefix. If X already has an edge c, nothing new is created. Otherwise create node cXc with length |X| + 2, and compute its suffix link by walking from link(X) to the next Y with s[i − |Y| − 1] = c and taking Y's c-child (for length 1 the link is the empty node). **Compute the link before inserting the new edge**; otherwise, when X is the −1 root, the walk can find the edge you just added and link the node to itself.

Complexity: use the depth of `last` in the suffix-link tree as a potential. Every link followed in the first walk decreases it by one, and the new `last` = cXc has depth at most depth(X) + 2 (its palindromic suffixes are c·Y·c for palindromic suffixes Y of X, plus "c"), so the first walks total O(n), as for the prefix function. A similar potential argument on the depth of link(last) bounds the second walk. Memory: 26 ints per node with an array; use `map` or a sorted edge list for large alphabets.

2.4 Counting occurrences

During construction increment `cnt[last]` after each character: `cnt[v]` counts positions where v is the *longest* palindromic suffix. All palindromes ending at position i are exactly the suffix-link chain from that `last`, so the number of occurrences of v is the sum of `cnt` over v's subtree in the link tree. Links point to nodes created earlier, so a single pass in reverse creation order (`cnt[link[v]] += cnt[v]`) propagates everything. APIO 2014 *Palindromes* ("maximise length × occurrences over palindromic substrings") is exactly this.

Other quantities available for free: number of distinct palindromes (node count − 2), number of palindromic substrings ending at each position (depth of `last` in the link tree, computed as `depth[v] = depth[link[v]] + 1`), and the longest palindromic suffix of every prefix.

2.5 Series links and O(n log n) palindromic DP

Lemma. If Q is a palindromic suffix of a palindrome P, then |P| − |Q| is a period of P. Consequently the lengths of the palindromic suffixes of any string split into O(log n) arithmetic progressions.

Sketch. Q is a suffix of P and, both being palindromes, also a prefix of P; so Q is a border and |P| − |Q| is a period. Define diff(v) = len(v) − len(link(v)). Along the suffix-link chain diff is non-increasing, and consecutive nodes with equal diff belong to one progression. When diff changes from d to d′ < d at node u, the periodicity lemma (§5) applied to the periods d and d′ of u shows that len(u) < d + d′ ≤ 2d, so the lengths at the boundaries of consecutive progressions at least halve. Hence O(log n) progressions.

Define the **series link** slink(v) as the first node on the link chain with diff different from diff(v). Walking via slinks visits O(log n) nodes per position.

Palindromic factorisation. dp[i] = minimal number of palindromes partitioning s[0, i). The naive recurrence dp[i] = min over palindromic suffixes P of s[0, i) of dp[i − |P|] + 1 can take Θ(n) per position. With progressions: for node v (on the slink chain of `last`), the progression consists of lengths len(v), len(v) − diff(v), …, len(slink(v)) + diff(v). Store `series[v]` = min of dp over the starting positions of that progression. The crucial observation is that the same progression, shifted by diff(v), was handled at position i − diff(v) at node link(v), so

series[v] = dp[i − (len(slink(v)) + diff(v))], combined with series[link(v)] if diff(v) = diff(link(v)).

Then dp[i] = min over the O(log n) nodes on the slink chain of series[v] + 1. This replaces min by any associative operation, which is how CF 932G (count even-length palindromic partitions after a transformation) and CF 906E (minimum reversals to turn one string into another) are solved.

3. Suffix trees

3.1 Definition and uses

The suffix tree of s is the compressed trie of all suffixes of s: every internal node except the root has at least two children, edges are labelled by substrings stored as (start, length) pairs. Appending a unique terminator `$` makes every suffix end at a leaf; the tree then has n leaves and at most 2n nodes. Uses:

- LCP of suffixes i and j = string depth of the LCA of their leaves.
- Number of distinct substrings = Σ of edge lengths (without the terminator).
- Longest repeated substring = deepest internal node; longest common substring of two strings = deepest node with leaves from both (generalised suffix tree).
- Matching statistics, k-mismatch matching with "kangaroo jumps" (O(1) LCP queries).

Ukkonen's algorithm builds the suffix tree online in O(n), but it is long and has subtle invariants. At contests two cheaper routes give the same structure.

3.2 Suffix tree from the suffix automaton of the reverse

Let t = reverse(s) and build its suffix automaton (nationals-06). A state v represents the set of substrings of t with the same set of end positions; they are the suffixes of the longest one, with lengths in (len(link v), len(v)]. Reversed back to s, these strings are the *prefixes* of a single string w_v, with the same lengths; and the parent link(v) represents prefixes of w_v of length ≤ len(link v). So the suffix-link tree is a trie of right-extensions in s, compressed at the points where the extension is no longer unique — that is exactly the suffix tree of s (whose nodes are the right-branching substrings and the suffixes).

To label edges, record for each SAM state the first end position `firstEnd` in t when the state is created (a clone copies it from the state it clones). An occurrence of the longest string of v in t ends at firstEnd(v), so w_v occurs in s starting at n − 1 − firstEnd(v), and the edge link(v) → v is labelled s[start + len(link v), start + len(v)).

3.3 Suffix tree from SA + LCP

The suffix array lists the leaves of the suffix tree in lexicographic order, and LCP[i] is the string depth of the LCA of adjacent leaves. The internal nodes are therefore the nodes of the Cartesian tree of the LCP array (with equal values merged), buildable with a monotone stack in O(n). This view is often enough without materialising the tree: CF 1073G answers queries "Σ LCP over pairs from two sets of suffixes" by building a virtual tree over the queried ranks with this stack and doing a small DP — the same technique as virtual trees in Lecture 4.

4. Lyndon words

4.1 Definitions and facts

A non-empty word w is **Lyndon** if it is strictly smaller than each of its proper rotations; equivalently, strictly smaller than each of its proper suffixes. Facts used constantly:

- A Lyndon word is *unbordered* (no proper prefix equals a suffix).
- If u and v are Lyndon and u < v, then uv is Lyndon.
- **Chen–Fox–Lyndon theorem:** every non-empty string factors uniquely as $w_1\, w_2 \cdots w_k$ with each w_i Lyndon and $w_1 \geq w_2 \geq \cdots \geq w_k$. The last factor w_k is the lexicographically smallest suffix of s; the first factor is the longest Lyndon prefix.

4.2 Duval's algorithm

Duval scans the string maintaining a decomposition $s = s_1\, s_2\, s_3$: s_1 is already factored; $s_2 = u^m u'$ where u is Lyndon, $m \geq 1$ and u' is a proper prefix of u (a "pre-Lyndon" string); s_3 is unread. Pointers: i = start of s_2, j = first character of s_3, $k = j - |u|$ (the character of the period that $s[j]$ is compared with).

- If $s[j] = s[k]$: the periodic structure continues; k++, j++.
- If $s[j] > s[k]$: s_2 followed by $s[j]$ is a Lyndon word (a larger character makes everything before it a single Lyndon word); set $k = i$ (the period becomes the whole thing), j++.
- If $s[j] < s[k]$: the copies of u are final factors. Emit $u, u, \ldots, u$ (m times) and restart at the beginning of u'.

Each restart moves i forward by at least the number of characters it re-reads, so the total time is $O(n)$ with fewer than $4n$ comparisons and $O(1)$ extra memory.

4.3 Minimal rotation and other uses

- **Least rotation**: run Duval on $s + s$; the answer is the start of the last factor that begins in the first half. The code in §7.3 is the standard compact version.
- **Smallest suffix** of every prefix and **largest suffix** computations follow from the same invariants.
- **Lyndon array** $\mathrm{Lyn}(i)$ = length of the longest Lyndon word starting at i = (next smaller suffix to the right) $- i$, computable from the inverse suffix array with a monotone stack. This is the ingredient for runs (§5.3).
- CF 594E *Cutting the Line* is a famous hard problem combining Lyndon factorisation of the reversed string with careful case analysis.

5. Periodicity

5.1 Periods and borders

An integer $p \in [1, n]$ is a **period** of s if $s[i] = s[i + p]$ for all $0 \leq i < n - p$. This holds iff s has a **border** (proper prefix equal to a suffix) of length $n - p$. All borders are obtained by iterating the prefix function from $\pi[n - 1]$, so all periods are known in $O(n)$. The smallest period is $n - \pi[n - 1]$; s is a proper power ($s = u^k$, $k \geq 2$) iff this p divides n and $p < n$.

5.2 Fine–Wilf and its consequences

Periodicity lemma (Fine–Wilf). If p and q are periods of s and $p + q - \gcd(p, q) \leq n$, then $\gcd(p, q)$ is a period of s.

Consequences:

- All periods $p \le n/2$ are multiples of the smallest period.
- The borders of a string (equivalently its periods) form $O(\log n)$ arithmetic progressions; the same holds for occurrences of a pattern P in a window of length $2|P|$ — they form one arithmetic progression.
- In the eertree, palindromic suffixes are borders of the longest palindromic suffix, which is why §2.5 works.

5.3 Runs

A **run** (maximal repetition) is a triple (l, r, p) such that $s[l, r)$ has smallest period p, $r - l \ge 2p$, and the period cannot be extended: $l = 0$ or $s[l-1] \ne s[l-1+p]$, and $r = n$ or $s[r] \ne s[r-p]$. Every square and every tandem repeat lies inside a run, so runs are a compact certificate for all repetitions.

Runs theorem (Bannai, I, Inenaga, Nakashima, Takeda, Tsuruta 2017). A string of length n has fewer than n runs, and the sum of exponents $(r-l)/p$ is less than $3n$.

The proof is also the algorithm. Consider two total orders on the alphabet, $<$ and its reverse. For a run (l, r, p), choose the order $\prec$ such that $s[r] \prec s[r-p]$ (if $r = n$, either order works with the convention that the end of the string is smallest). A **Lyndon root** of the run is a substring of length p inside the run that is $\prec$-Lyndon. The key lemma: if $s[i, i+p)$ is a Lyndon root of the run with respect to $\prec$, then it is the *longest* $\prec$-Lyndon word starting at i. Distinct runs have Lyndon roots starting at distinct (position, order) pairs, which gives the bound $< n$ (with care about position l).

Algorithm: for each order, compute the Lyndon array (next $\prec$-smaller suffix with a monotone stack); for each i let $j = i + \mathrm{Lyn}(i)$, $p = j - i$. Extend the period p: L = longest common suffix of $s[0, i)$ and $s[0, j)$, R = longest common prefix of suffixes i and j. If $L + R \ge p$, then $(i - L, j + R, p)$ is a run. Collect and deduplicate. With hashing and binary search the LCP/LCS queries cost $O(\log n)$ and suffix comparisons in the stack also $O(\log n)$, so the whole algorithm runs in $O(n \log n)$ (about 2.4 s for $n = 10^6$ in our test with a 61-bit modular hash — use SA + sparse table for $O(1)$ LCP when speed matters).

6. How these appear in contest problems

- "Count/maximise something over all distinct palindromic substrings" → eertree with occurrence counts.
- "Partition into palindromes with minimum/count/…" with n up to 10^6 → series links.
- "Lexicographically smallest rotation / cyclic string canonical form" → Duval or the two-pointer minimal rotation; also used to canonicalise necklaces before hashing (combine with Lecture 7).

- "Count squares / tandem repeats / substrings of the form A^k" → runs; e.g. the number of distinct squares is O(n), and all squares can be enumerated as sliding windows of length 2p inside runs.
- "Many LCP queries between arbitrary suffixes" → suffix tree (or SA + RMQ); sets of suffixes → virtual tree over the suffix tree.

7. Reference implementations

7.1 Eertree — APIO 2014 Palindromes

Input: a string of lowercase letters ($|s| \le 3 \cdot 10^5$). Output: the maximum over palindromic substrings of length × number of occurrences.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct Eertree {
    struct Node { int len, link, cnt, diff, slink; array<int, 26> nxt; };
    vector<Node> t; string s; int last;
    int newNode(int len, int link) { t.push_back({len, link, 0, 0, 0, {}}); return
(int)t.size() - 1; }
    Eertree() { newNode(-1, 0); newNode(0, 0); last = 1; }   // 0: root of length -1, 1:
empty string
    int getLink(int v, int i) const {
        while (i - 1 - t[v].len < 0 || s[i - 1 - t[v].len] != s[i]) v = t[v].link;
        return v;
    }
    bool add(char ch) {
        s.push_back(ch);
        int i = (int)s.size() - 1, c = ch - 'a';
        int cur = getLink(last, i);
        bool created = false;
        if (!t[cur].nxt[c]) {
            int len = t[cur].len + 2;
            int link = len == 1 ? 1 : t[getLink(t[cur].link, i)].nxt[c];   // before
inserting the edge
            int v = newNode(len, link);
            t[v].diff = len - t[link].len;
            t[v].slink = t[v].diff == t[link].diff ? t[link].slink : link;
            t[cur].nxt[c] = v;
            created = true;
        }
        last = t[cur].nxt[c];
        t[last].cnt++;
        return created;
    }
    void propagate() { for (int v = (int)t.size() - 1; v >= 2; v--) t[t[v].link].cnt +=
t[v].cnt; }
};
```

```
int main() {
    string s;
    cin >> s;
    Eertree T;
    for (char c : s) T.add(c);
    T.propagate();
    ll best = 0;
    for (int v = 2; v < (int)T.t.size(); v++) best = max(best, (ll)T.t[v].len *
T.t[v].cnt);
    cout << best << '\n';
}
```

7.2 Minimum palindromic factorisation in O(n log n) (uses `Eertree` above)

```
vector<int> palFactor(const string& s) {       // dp[i] = min #palindromes covering s[0,
i)
    int n = s.size();
    Eertree T;
    vector<int> dp(n + 1, INT_MAX / 2), series(n + 3, INT_MAX / 2);
    dp[0] = 0;
    for (int i = 1; i <= n; i++) {
        T.add(s[i - 1]);
        series.resize(T.t.size(), INT_MAX / 2);
        for (int v = T.last; T.t[v].len > 0; v = T.t[v].slink) {
            series[v] = dp[i - (T.t[T.t[v].slink].len + T.t[v].diff)];
            if (T.t[v].diff == T.t[T.t[v].link].diff) series[v] = min(series[v],
series[T.t[v].link]);
            dp[i] = min(dp[i], series[v] + 1);
        }
    }
    return dp;
}
```

Tested against the O(n³) DP for all prefixes of 3000 random strings; n = 10^6 binary string runs in well under a second.

7.3 Duval and minimal rotation

```
vector<pair<int, int>> duval(const string& s) {   // (start, length) of each factor, non-
increasing
    vector<pair<int, int>> res;
    int n = s.size(), i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) { k = (s[k] < s[j]) ? i : k + 1; j++; }
        while (i <= k) { res.push_back({i, j - k}); i += j - k; }
    }
    return res;
```

```
}
int minRotation(string s) {                         // smallest index of the least
rotation
    int n = s.size(), i = 0, ans = 0;
    s += s;
    while (i < n) {
        ans = i;
        int j = i + 1, k = i;
        while (j < 2 * n && s[k] <= s[j]) { k = (s[k] < s[j]) ? i : k + 1; j++; }
        while (i <= k) i += j - k;
    }
    return ans;
}
```

`minRotation` returns the *smallest* starting index among equal least rotations (checked against brute force).

7.4 Suffix tree via SAM of the reversed string

```
struct SuffixTree {
    struct St { int len, link, firstEnd; map<char, int> nx; };
    vector<St> st; int n; string s;
    vector<vector<int>> ch;     // children in the suffix tree (root = 0)
    vector<int> start;          // node v spells s[start[v], start[v] + len[v])
    SuffixTree(const string& s_) : n(s_.size()), s(s_) {
        string t(s.rbegin(), s.rend());
        st.push_back({0, -1, -1, {}});
        int last = 0;
        for (int i = 0; i < n; i++) {
            char c = t[i];
            int cur = st.size();
            st.push_back({st[last].len + 1, -1, i, {}});
            int p = last;
            while (p != -1 && !st[p].nx.count(c)) st[p].nx[c] = cur, p = st[p].link;
            if (p == -1) st[cur].link = 0;
            else {
                int q = st[p].nx[c];
                if (st[p].len + 1 == st[q].len) st[cur].link = q;
                else {
                    int cl = st.size();
                    st.push_back(st[q]);            // clone keeps q's firstEnd
                    st[cl].len = st[p].len + 1;
                    while (p != -1 && st[p].nx[c] == q) st[p].nx[c] = cl, p = st[p].link;
                    st[q].link = st[cur].link = cl;
                }
            }
            last = cur;
        }
        int m = st.size();
        ch.assign(m, {});
        start.assign(m, 0);
        for (int v = 1; v < m; v++) {
```

```
            ch[st[v].link].push_back(v);
            start[v] = n - 1 - st[v].firstEnd;
        }
    }
    string edge(int v) const {   // label of the edge parent(v) -> v
        int p = st[v].link;
        return s.substr(start[v] + st[p].len, st[v].len - st[p].len);
    }
};
```

Validation: for random strings, DFS over the tree shows that children of each node start with distinct characters and that the set of all strings spelled along root paths (including partial edges) is exactly the set of distinct substrings of s.

7.5 All runs via Lyndon roots

```
struct Hash {
    static const unsigned long long M = (1ULL << 61) - 1;
    vector<unsigned long long> h, pw;
    static unsigned long long mul(unsigned long long a, unsigned long long b) {
        __uint128_t c = (__uint128_t)a * b;
        c = (c & M) + (c >> 61);
        return c >= M ? c - M : c;
    }
    Hash(const string& s, unsigned long long B = 911382323) : h(s.size() + 1, 0),
pw(s.size() + 1, 1) {
        for (size_t i = 0; i < s.size(); i++) {
            h[i + 1] = (mul(h[i], B) + (unsigned char)s[i]) % M;
            pw[i + 1] = mul(pw[i], B);
        }
    }
    unsigned long long get(int l, int r) const { return (h[r] + M - mul(h[l], pw[r - l]))
% M; }  // [l, r)
};
struct Run {
    int l, r, p;   // s[l, r) has smallest period p, r - l >= 2p, maximal
    bool operator<(const Run& o) const { return tie(l, r, p) < tie(o.l, o.r, o.p); }
};
vector<Run> runs(const string& s) {
    int n = s.size();
    Hash H(s);
    auto lcp = [&](int i, int j) {
        int lo = 0, hi = n - max(i, j);
        while (lo < hi) { int m = (lo + hi + 1) / 2; if (H.get(i, i + m) == H.get(j, j +
m)) lo = m; else hi = m - 1; }
        return lo;
    };
    auto lcs = [&](int i, int j) {   // common suffix of s[0, i) and s[0, j)
        int lo = 0, hi = min(i, j);
        while (lo < hi) { int m = (lo + hi + 1) / 2; if (H.get(i - m, i) == H.get(j - m,
j)) lo = m; else hi = m - 1; }
        return lo;
```

```
    };
    set<Run> res;
    for (int inv = 0; inv < 2; inv++) {
        auto less = [&](int i, int j) {   // suffix i < suffix j; a proper prefix is
smaller
            int l = lcp(i, j);
            if (i + l == n) return true;
            if (j + l == n) return false;
            return inv ? s[i + l] > s[j + l] : s[i + l] < s[j + l];
        };
        vector<int> stk;
        for (int i = n - 1; i >= 0; i--) {
            while (!stk.empty() && !less(stk.back(), i)) stk.pop_back();
            int j = stk.empty() ? n : stk.back();     // longest Lyndon word at i is s[i,
j)
            int p = j - i, L = lcs(i, j), R = j < n ? lcp(i, j) : 0;
            if (j < n && L + R >= p) res.insert({i - L, j + R, p});
            stk.push_back(i);
        }
    }
    return vector<Run>(res.begin(), res.end());
}
```

Validated against an $O(n^3)$ enumeration of maximal repetitions on 3000 random strings over alphabets of size 1–3. Example: `aabaabaab` has the runs [0,2) p=1, [3,5) p=1, [6,8) p=1 and [0,9) p=3.

8. Common mistakes

1. **Eertree link computed after inserting the edge.** For a new single character, the walk from link(−1 root) reaches the −1 root, whose c-edge you just created: the node links to itself and later walks loop forever.
2. **Holding `Node&` across `push_back`.** The vector reallocates; use indices.
3. **Propagating counts in the wrong order.** Links point to earlier-created nodes, so reverse creation order is a valid topological order; sorting by length also works, but node index order is simpler and correct.
4. **Using the eertree with a larger alphabet but `array<int,26>`.** Map characters first or switch to `map`.
5. **SAM clones not copying `firstEnd`.** The clone represents a subset of the original's strings with the same end positions; it must inherit them.
6. **Duval: emitting the factor only once.** The inner loop emits several equal Lyndon factors (e.g. `abab` → `ab`, `ab`).
7. **Runs: forgetting to deduplicate**, or using only one of the two orders (runs whose following character is larger are found only with the reversed order).
8. **Assuming Fine–Wilf without the length condition.** `aabaa` has periods 3 and 4, but $p + q - \gcd(p, q) = 6 > 5$ and indeed $\gcd = 1$ is not a period.

9. Practice problems

Problem	Technique
APIO 2014 — Palindromes (oj.uz)	eertree, occurrence counts
Timus 1960 — Palindromes and Super Abilities	eertree online
CF 17E — Palisection	palindrome counting (Manacher or eertree)
CF 932G — Palindrome Partition	series links, counting DP
CF 906E — Reverses	series links, min DP with reconstruction
UVa 719 — Glass Beads (vjudge)	minimal rotation
SPOJ MINMOVE — Minimum Rotations	minimal rotation
Library Checker — Lyndon Factorization	Duval
Library Checker — Run Enumerate	runs
CF 594E — Cutting the Line	Lyndon factorisation, hard
CF 526D — Om Nom and Necklace	periods via prefix function / Z
CF 432D — Prefixes and Suffixes	borders and their occurrence counts
CF 1073G — Yet Another LCP Problem	suffix-tree structure from SA + LCP, virtual tree

10. Further reading

- M. Rubinchik, A. M. Shur, "EERTREE: An efficient data structure for processing palindromes in strings" (European Journal of Combinatorics, 2018).
- M. Crochemore, C. Hancart, T. Lecroq, *Algorithms on Strings* (Cambridge University Press, 2007) — periodicity, Lyndon words, suffix trees.
- D. Gusfield, *Algorithms on Strings, Trees, and Sequences* (1997) — suffix trees and their applications.
- J.-P. Duval, "Factorizing words over an ordered alphabet" (J. Algorithms, 1983).
- H. Bannai, T. I, S. Inenaga, Y. Nakashima, M. Takeda, K. Tsuruta, "The 'Runs' Theorem" (SIAM J. Computing, 2017).
- cp-algorithms: "Lyndon factorization", "Suffix Tree. Ukkonen's Algorithm" — https://cp-algorithms.com/